

AI 面试官语音转录项目

阶段性汇报、整体方案设计、技术路线与部署说明

汇报日期：2026-08-19

项目仓库：RichardF123/AI-interviewer-speech-recognition

汇报重点	结论
项目定位	电话式 AI 面试语音承载层，目标是让候选人像打电话一样说话，系统实时转写，大模型理解后立即以文字和语音反馈。
当前阶段	已经完成可本地运行 Demo：前端自动开麦、实时/准实时字幕、WebSocket 会话、DeepSeek 回复、TTS 播放和打断控制雏形。
核心问题	现在不是内容层面问题，而是实时语音链路稳定性和端到端延迟问题：ASR、LLM streaming、TTS streaming 必须同时优化。
下一步	把架构从“可演示”升级到“稳定实时”：流式 ASR、流式 LLM、流式 TTS、VAD、回声消除、provider 健康检查。

1. 一句话方案

本项目不是做一个普通聊天页，而是做一个可装载任意面试大模型的实时语音交互层。候选人进入面试后无需反复点录音按钮，系统持续监听候选人说话；识别到有效回答后实时显示文字，将稳定 final 文本送入大模型；大模型输出后立刻展示文字并播放语音。

2. 端到端链路

候选人麦克风 -> 前端音频采集 -> 实时 ASR partial 字幕 -> turn end/final 判定 -> WebSocket 发送给后端 -> LLM 生成 -> AI 文本输出 -> TTS 合成 -> 前端播放 -> 自动回到监听。

输入/输出	事件	说明
候选人说话	audio.chunk	前端把麦克风音频按小片段送到后端，当前目标格式为 16k PCM。
实时字幕	stt.partial	只用于屏幕即时显示，不进入 LLM，避免半句话触发错误追问。
稳定回答	stt.final / speech.final	只有 final 文本进入 LLM；系统状态文本会被拦截。
模型理解	llm.input / assistant.text	候选人 final 文本进入 DeepSeek 或未来微调模型，生成 AI 面试官追问。
语音播放	tts.audio / Browser TTS	优先真实 TTS，失败时浏览器 TTS 兜底，保证 demo 可听见。
打断控制	control.interrupt	候选人插话时取消当前播放/生成，恢复监听。

3. 设计原则

- ☐ 前端不展示复杂卡片，把核心体验压缩为：AI 输出区、候选人实时输入区、状态与延迟指标。
- ☐ partial 只负责“马上看到自己说的话”，final 才负责“让大模型理解”。
- ☐ 所有 provider 都通过后端编排，前端只处理麦克风、字幕、播放和控制。
- ☐ 大模型必须可替换，后续可直接接入微调后的 OpenAI-compatible 模型。
- ☐ 端到端目标不是单点优化，而是 ASR、LLM、TTS 三段同时 streaming。

4. 整体方案设计架构

层级	职责	当前实现	下一阶段目标
前端交互层	电话式面试界面、自动开麦、实时字幕、播放 AI 声音	React + Vite + TypeScript; Web Speech API 显示 partial/final; WebSocket 连接后端; Browser TTS 兜底	AudioWorklet 替代 ScriptProcessor; 加入 VAD、打断、回声抑制和状态可观测面板
语音输入层	把候选人麦克风音频转为可理解文本	Web Audio API 采集 16k PCM; 浏览器 SpeechRecognition 作为低延迟字幕; 阿里云 NLS 实时 ASR 作为后端主链路; MiMo 作为分段备用	稳定后端流式 ASR partial/final, 取消对浏览器 ASR 的强依赖
会话编排层	管理一轮对话的顺序、去重、打断和事件协议	FastAPI WebSocket; 只允许 stt.final/speech.final 进入 LLM; 拦截系统状态文本; 最新输入优先取消上一轮任务	统一 Turn State Machine: listening -> finalizing -> thinking -> speaking -> interrupted
LLM 层	把候选人最终文本送入大模型并生成面试官回复	DeepSeek Chat Completions, 当前以整段回复为主; prompt 定位为语音面试承载层	Streaming token 输出; 可替换为微调后的 OpenAI-compatible 模型
语音输出层	把 AI 回复转换为自然语音并播放	阿里云 NLS TTS + 浏览器 speechSynthesis 兜底; 前端优先确保可听见	句子级流式 TTS、播放队列、低延迟首包、支持候选人插话打断
部署运维层	本地运行、API 配置、日志和上线准备	后端 .env 管理 API; 前端 VITE_API_BASE; GitHub 已有代码和阶段报告	前端静态托管 + 后端云服务 + WSS/HTTPS + 日志监控 + 密钥托管

架构判断

当前 Demo 采用“前端实时体验优先、后端 provider 编排兜底”的策略。这样能最快跑通电话式交互，但要达到接近豆包电话的体验，下一阶段必须让后端 ASR partial 稳定返回，并把 LLM 与 TTS 改为真正流式。

5. 技术路线

阶段	目标	关键技术	交付标准
MVP 可演示	本地跑通电话式 AI 面试 Demo	React/Vite、FastAPI、WebSocket、浏览器 SpeechRecognition、DeepSeek、阿里云 TTS	开始面试后能自动监听，说话能显示，AI 能回复并朗读。
实时稳定版	说完 2-3 秒内 AI 理解并反馈	阿里云实时 ASR、AudioWorklet、VAD、DeepSeek streaming、句子级 TTS 队列	partial 低延迟、final 稳定、首字/首包可观测，连续多轮不卡顿。
生产可集成版	接入微调模型与正式面试业务	Provider 抽象、Session 状态机、日志追踪、鉴权、HTTPS/WSS、密钥托管	可部署到云端，前端可嵌入业务系统，后端可切换模型和语音厂商。
电话级体验版	接近豆包电话的自然交互	全双工流式语音、回声消除、barge-in、低延迟合成、实时指标报警	AI 说话时用户可插话，系统能立即中断并理解新输入。

6. 2-3 秒延迟目标拆解

链路节点	目标时延	当前策略	改进动作
麦克风到 partial 字幕	100-500ms	浏览器 SpeechRecognition 先显示，后端同时收 PCM	改 AudioWorklet 10-20ms 分片，ASR ready 前缓存音频
停顿到 final	300-900ms	前端检测 1.2s 停顿后提交 final	引入 VAD/endpointer，按语音能量和静音窗口判定 turn end
final 到 LLM 首字	500-1200ms	DeepSeek 完整回复后返回	开启 streaming，收到首 token 立刻展示
LLM 到 TTS 首包	500-1000ms	完整文本后合成或浏览器 TTS 兜底	按句切分，边生成边合成，播放队列预热
总体验目标	2-3s	Demo 可跑，但依赖浏览器 ASR 和 provider 可用性	全链路 streaming + VAD + provider 健康检查

7. 技术栈与解决的问题

技术栈	作用	解决的问题
React + Vite + TypeScript	构建前端 Demo 与电话式交互界面	快速实现可演示、可迭代、类型相对可控的前端。
Web Audio API	采集麦克风音频并转为 PCM	把真实候选人声音送入后端 ASR，而不是只靠文本输入。
Browser SpeechRecognition	前端实时字幕兜底	在后端 ASR 不稳定时，仍让用户马上看到自己说的话。
FastAPI + WebSocket	语音会话网关和事件流	支持双向实时通信，比普通 HTTP 更适合语音交互。
DeepSeek Chat Completions	AI 面试官文本生成	让回复来自真实大模型，不再是固定 mock 文案。
阿里云 NLS ASR/TTS	实时语音识别和语音合成	承担生产方向上的中文语音输入/输出能力。
MiMo ASR	分段语音识别备份	当实时 ASR 不可用时，可做录音片段转写备用。
GitHub	版本管理和阶段交付	保留每次架构、代码、文档变更记录，方便上线和回滚。

8. 当前情况

- ☐ 前端页面已从复杂卡片改成电话式主界面：上方展示 AI 面试官输出，下方展示候选人实时输入。
- ☐ 点击开始面试后会请求麦克风权限，并尝试自动进入持续监听。
- ☐ 候选人 `partial` 文本只在屏幕显示，不会误当成正式回答。
- ☐ `final` 文本会进入后端，并由 DeepSeek 生成 AI 面试官回复。
- ☐ AI 回复会触发语音播报；阿里云 TTS 不稳定时使用浏览器 TTS 兜底。
- ☐ 已加入多轮输入优先级处理，避免上一轮 LLM/TTS 卡住下一轮输入。
- ☐ 当前最大风险：阿里云实时 ASR 在本机曾出现 WebSocket 启动超时，导致后端 `partial` 不稳定；浏览器 `SpeechRecognition` 在 Codex 内置浏览器中可能不可用，建议用 Chrome/Edge 验证。
- ☐ 代码/文档存在中文编码显示异常迹象，需要单独安排一次 UTF-8 编码治理。

9. 需要的 API 服务

服务	必需配置	用途
DeepSeek	DEEPSEEK_API_KEY、DEEPSEEK_BASE_URL、DEEPSEEK_MODEL	AI 面试官文本理解与回复。
阿里云 NLS	ALIYUN-NLS_APP_KEY、ALIYUN_ACCESS_KEY_ID、ALIYUN_ACCESS_KEY_SECRET、ALIYUN-NLS_TOKEN、ALIYUN-NLS_ENDPOINT	实时 ASR 和 TTS。需要确认账号权限、项目 AppKey、地域 endpoint 与 token 有效期。
MiMo	MIMO_API_KEY、MIMO_BASE_URL、MIMO_ASR_MODEL、MIMO_ASR_LANGUAGE	分段 ASR 备用，不建议作为实时电话体验主链路。
前端配置	VITE_API_BASE	指定后端 API 地址，本地默认 <code>http://127.0.0.1:8011</code> 。

密钥管理要求：真实 Key 只放在本地 `.env` 或云端 Secret Manager，不能写入 README、前端代码或 GitHub。

10. 部署路径

步骤	操作	说明
1. 准备密钥	在 app/backend/.env 填 DEEPSEEK_API_KEY、ALIYUN_NLS_APP_KEY、ALIYUN_ACCESS_KEY_ID、ALIYUN_ACCESS_KEY_SECRET、ALIYUN_NLS_TOKEN、MIMO_API_KEY	真实密钥不能进入 GitHub；生产环境应使用云厂商 Secret Manager
2. 启动后端	cd app/backend; 创建 venv; pip install -r requirements.txt; uvicorn app.main:app --host 127.0.0.1 --port 8011	后端提供 REST 会话创建、provider 状态检查和 WebSocket 语音会话
3. 启动前端	cd app/frontend; npm install; npm run dev -- --host 127.0.0.1 --port 5185	浏览器访问 http://127.0.0.1:5185/；建议 Chrome/Edge
4. 验证链路	打开 /health 和 /api/provider-status；开始面试后允许麦克风；说话看 partial；停顿后看 AI 回复和语音播放	重点观察 STT final、LLM 首响、TTS 首包、端到端时间
5. 上线方案	前端部署到静态 CDN；后端部署到云服务器/容器服务；域名启用 HTTPS/WSS；环境变量走云端密钥配置	浏览器麦克风在生产环境要求 HTTPS，WebSocket 也应使用 WSS

11. 上线架构建议

生产部署建议采用：前端 CDN/静态托管 + 后端容器服务 + HTTPS/WSS + 云端 Secret Manager + 日志监控。浏览器麦克风权限在生产环境通常要求 HTTPS，因此正式演示和上线不能长期依赖 http 页面。

推荐生产链路：

用户浏览器 Chrome/Edge -> HTTPS 前端 -> WSS 后端语音网关 -> 阿里云实时 ASR -> DeepSeek/微调模型 streaming -> 阿里云流式 TTS -> 前端播放队列。

12. 下一阶段交付清单

- ☐ P0: 修复中文编码问题，保证 README、前端文案和日志在 GitHub/本地都正常显示。
- ☐ P0: 稳定阿里云实时 ASR，确认 token、AppKey、地域 endpoint、账号权限和本机网络连通。
- ☐ P0: DeepSeek 改为 streaming 输出，前端收到首 token 立即显示。
- ☐ P1: 把 ScriptProcessor 升级为 AudioWorklet，降低麦克风分片延迟。
- ☐ P1: 加入 VAD/endpointer，减少“说完后等待太久”的问题。
- ☐ P1: TTS 改为句子级流式合成与播放队列，降低 AI 首声音频延迟。
- ☐ P1: 加入 barge-in，用户插话时立即停止 AI 播放并进入新一轮识别。
- ☐ P2: 抽象 Provider 接口，支持后续接入微调大模型或端到端实时语音模型。